

Checkpoint 3.1 — Pipeline Dependencies, Cold Start & Client Archetypes

Amen Bank — Real-Time Anomaly Detection Platform

Internship Project Documentation

June 3, 2026

1 Purpose

This checkpoint captures three related decisions that emerged while preparing to design the client and employee profile schemas: the dependency relationship between the batch and streaming pipelines, the cold-start strategy for new clients, and the formal definition of the five client archetypes that serve as proxy profiles.

This document builds on Checkpoint 3 (raw event schema) and feeds directly into the upcoming profile schema design.

2 Pipeline Dependency Chain

2.1 The Question

Before implementing services, we asked: should we build the streaming pipeline (Ingestion → Enrichment → Scoring → Decision) first, or the batch pipeline?

2.2 Analysis

The streaming pipeline has two hard dependencies that only the batch pipeline can satisfy:

1. **Profiles.** The Enrichment Service looks up client and employee profiles in Redis (hot store). These profiles are computed by the batch pipeline from accumulated raw transactions in PostgreSQL.
2. **Trained models.** The Scoring Service requires a trained Autoencoder and LSTM. These models are trained by the batch pipeline on historical feature vectors.

2.3 Circular Dependency — Debunked

An initial concern was that the batch pipeline might depend on Redis (populated by the streaming pipeline), creating a circular dependency. This was resolved by clarifying that PostgreSQL stores **two kinds of data**:

- **Raw transaction records** — every event persisted as it flows through the system.
- **Computed cold profiles** — aggregate statistics rebuilt nightly by the batch pipeline.

The batch pipeline reads from **raw transaction records**, not from Redis. The dependency is therefore **linear, not circular**:

Raw events → PostgreSQL (transaction store) → Batch pipeline → Profiles
(PostgreSQL + Redis) → Streaming pipeline reads from Redis

2.4 Decision: Batch Pipeline First

Implementation order: build the batch pipeline (synthetic data generation → feature engineering → profile computation → model training) before the streaming pipeline.

3 Cold-Start Strategy

3.1 The Problem

When a **brand-new client** makes their very first transaction, no personal profile exists in Redis. The Enrichment Service cannot compute behavioral deviation features. This is the **cold-start problem**.

A key concern: what if the new client’s first transaction is itself fraudulent? Without history, personal deviation detection is blind.

3.2 Three Layers of Defense (No Profile Required)

Even with zero personal history, the system retains three independent detection mechanisms:

1. **Regulatory rules and thresholds.** Hard rules from LAB/FT regulations (Banque Centrale de Tunisie) apply to every transaction regardless of client identity. Examples: the 10 000 DT reporting threshold, smurfing pattern detection (multiple deposits just below threshold), duplicate transaction detection, business-hours constraints.
2. **Archetype baseline comparison.** Even without personal history, the bank knows the client’s archetype from KYC and account-opening data. The incoming transaction is compared against the population-level baseline for that archetype. A new student account making a 50 000 DT versement immediately deviates from the student baseline.
3. **Employee profile (dual-actor model).** The employee processing the transaction has their own profile. Suspicious employee-side patterns (high error rate, near-threshold clustering, client-location mismatches) are detected independently of the client’s history.

Once the batch pipeline runs and accumulates enough transaction history, the fourth layer activates: **personal behavioral deviation** detection via the client’s own profile.

4 Client Archetypes

Five archetypes serve as population-level baselines and cold-start proxy profiles. Each archetype defines expected “normal” behavior across several dimensions.

Archetype	Deposit Pattern	Amounts	Primary Operations	Typical Hours
Student	Irregular (parents send money at random times)	Small	Withdrawal (dominant)	Evening / lunch time

Archetype	Deposit Pattern	Amounts	Primary Operations	Typical Hours
Salaried	Regular, beginning of month (salary); stable amount with gradual increases over time	Moderate, stable	Withdrawal (dominant); occasional transfers (debt payments)	Start of day (morning)
Small Business	Irregular, varied amounts, never extremely high	Moderate, variable	All four operations (withdrawal, deposit, transfer, cheque)	Random / business hours
Retiree	Beginning of month only; constant amount (pension)	Small to moderate, very stable	Withdrawal (dominant)	Morning
Big Business	Irregular, varied amounts, can be very large	Large to very large	All four operations	Random / business hours

Key insight: Retiree is the most predictable archetype (fixed pension, fixed schedule), making any deviation highly conspicuous. Student and small business are the noisiest, requiring wider tolerance bands.

5 Impact on Upcoming Design

1. **Client profile schema** must include an **archetype** field and the profile must gracefully handle the cold-start case (fallback to archetype baseline when personal statistics are insufficient).
2. **Batch pipeline** must compute and maintain archetype-level aggregate baselines in addition to per-client profiles.
3. **Synthetic data generator** must produce realistic transaction distributions for each archetype, respecting the behavioral patterns defined above.
4. **Implementation order** is batch-first: synthetic data → batch feature engineering → profile computation → model training → then streaming services.

6 Open Questions

1. **Cold-start graduation.** How many transactions (or days of history) before the system trusts the personal profile over the archetype baseline? Need a blending strategy.

2. **Archetype migration.** A student graduates and becomes salaried. How and when does the archetype label update?
3. **Archetype baseline refresh.** How often are the population-level baselines recomputed? Nightly with profile rebuild, or on a slower cadence?

7 Next Step

Design the **client profile schema** (hot in Redis, cold in PostgreSQL) and the **employee profile schema**, defining the exact fields, rolling windows, and how the Enrichment Service uses them to produce the feature vector for the model.